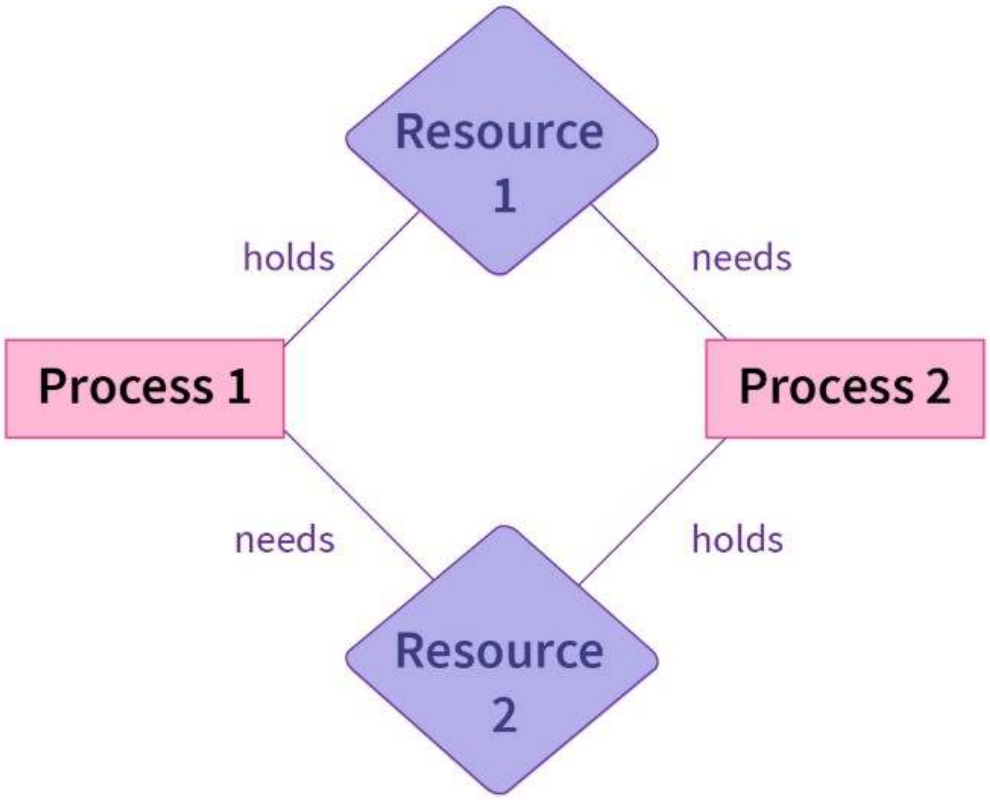
What is Deadlock in OS?

All the processes in a system require some resources such as central processing unit (CPU), file storage, input/output devices, etc to execute it. Once the execution is finished, the process releases the resource it was holding. However, when many processes run on a system they also compete for these resources they require for execution. This may arise a deadlock situation.

A deadlock is a situation in which more than one process is blocked because it is holding a resource and also requires some resource that is acquired by some other process. Therefore, none of the processes gets executed.

Example of Deadlock:



In the above figure, there are two processes and two resources. Process 1 holds "Resource 1" and needs "Resource 2" while Process 2

holds "Resource 2" and requires "Resource 1". This creates a situation of deadlock because none of the two processes can be executed. Since the resources are non-shareable they can only be used by one process at a time (Mutual Exclusion). Each process is holding a resource and waiting for the other process the release the resource it requires. None of the two processes releases their resources before their execution and this creates a circular wait. Therefore, all four conditions are satisfied.

Necessary Conditions for Deadlock:

The four necessary conditions for a deadlock to arise are as follows.

- **Mutual Exclusion**: Only one process can use a resource at any given time i.e. the resources are non-sharable.

- **Hold and wait**: A process is holding at least one resource at a time and is waiting to acquire other resources held by some other process.

- **No pre-emption**: The resource can be released by a process voluntarily i.e. after execution of the process.

- **Circular Wait**: A set of processes are waiting for each other in a circular fashion. For example, lets say there are a set of processes {P0*P*0,P1*P*1,P2*P*2,P3*P*3} such that P0*P*0 depends on P1*P*1 , P1*P*1 depends on P2*P*2, P2*P*2 depends on P3*P*3 and P3*P*3 depends on P0*P*0. This creates a circular relation between all these processes and they have to wait forever to be executed.

Methods of Handling Deadlocks in Operating System:

The first two methods are used to ensure the system never enters a deadlock.

Deadlock Prevention

This is done by restraining the ways a request can be made. Since deadlock occurs when all the above four conditions are met, we try to prevent any one of them, thus preventing a deadlock.

Deadlock Avoidance

Deadlock avoidance is a strategy used by the operating system to ensure the system **never enters an unsafe state**.
When a process requests a resource, the **deadlock avoidance algorithm** (like the **Banker's Algorithm**) examines the **current resource-allocation state**.
If granting the resource would **lead to a potentially unsafe state**, the OS **denies or delays** the request.

Deadlock Detection and Recovery

We let the system fall into a deadlock and if it happens, we detect it using a detection algorithm and try to recover.

Some ways of recovery are as follows.

- Aborting all the deadlocked processes.
- Abort one process at a time until the system recovers from the deadlock.
- Resource Pre-emption: Resources are taken one by one from a process and assigned to higher priority processes until the deadlock is resolved.

Deadlock Ignorance

In the method, the system assumes that deadlock never occurs. Since the problem of deadlock situation is not frequent, some systems

simply ignore it. Operating systems such as UNIX and Windows follow this approach. However, if a deadlock occurs we can reboot our system and the deadlock is resolved automatically.